

Selenium UI Automation using Java (Amazon Filter & Sorting Validation)

Objective

Implemented a Selenium UI automation framework using Java, JUnit 5, and the Page Object Model (POM) design pattern to automate the Amazon product search, filtering, sorting, and product validation workflow.

Tasks Completed

1. Project Structure Setup

Created a modular Selenium automation framework with the following package structure:

- **base**
 - BaseTest.java
- **Pages**
 - HomePage.java
 - SearchResultPage.java
 - ProductPage.java
- **tests**
 - AmazonTest.java
- **utils**
 - DriverFactory.java

This structure separates page actions, test logic, and browser configuration, making the framework reusable and maintainable.

2. Base Test Configuration

Configured the common test setup and teardown methods.

Implemented:

- Browser initialization using DriverFactory.
- Opening Amazon India URL.
- Browser maximization.

- Closing the browser after test execution.

This eliminates duplicate setup code across test classes.

3. Home Page Automation

Implemented the HomePage class following the Page Object Model.

Developed functionality to:

- Locate the Amazon search box.
 - Search for the product "**Headphones**".
 - Submit the search using the Enter key.
-

4. Search Results Page Automation

Created the SearchResultPage to handle all product listing interactions.

Implemented methods for:

- Selecting the **GOBOULT** brand filter.
- Applying **Price: Low to High** sorting.
- Capturing displayed product prices.
- Clicking the first available product.

Applied Explicit Waits to synchronize with dynamically loaded elements.

5. Product Page Automation

Created ProductPage to validate product details.

Implemented functionality to:

- Retrieve the product title.
 - Verify that the selected product belongs to the GOBOULT brand.
-

6. Test Case Implementation

Developed an end-to-end UI automation test covering the complete business scenario.

Workflow automated:

1. Launch Amazon India.
2. Search for "Headphones".
3. Apply GOBOULT brand filter.
4. Apply Price: Low to High sorting.
5. Capture product prices.
6. Validate prices are sorted in ascending order.
7. Open the first product.
8. Verify that the product belongs to GOBOULT.

Selenium Concepts Used

- Selenium WebDriver
- Java
- JUnit 5
- Page Object Model (POM)
- Explicit Wait (WebDriverWait)
- ExpectedConditions
- JavaScriptExecutor
- Select class
- List Collection
- Collections.sort()
- Assertions
- Window Handling
- Dynamic XPath
- CSS Selectors

Challenges Faced

Dynamic Amazon UI

Amazon frequently changes its HTML structure, causing locators to become unstable.

Dynamic Loading

Product results load asynchronously after filters and sorting are applied, requiring synchronization using Explicit Waits.

Product Locator Issues

The generic locator for the first product (h2 a) was not reliable because Amazon includes sponsored products, advertisements, and recommendation sections.

Timeout Exceptions

Encountered TimeoutExceptions while locating product prices and clicking the first product due to changing DOM structures and dynamic page updates.

Debugging & Fixes Performed

- Updated product and filter locators.
- Replaced generic locators with more specific XPath expressions.
- Improved synchronization using Explicit Waits.
- Used JavaScriptExecutor for reliable scrolling and clicking.
- Implemented browser window switching after opening the selected product.
- Added exception handling while reading product prices.
- Refined locator strategies for better stability.

Current Status

- Project framework successfully created.
- Page Object Model implemented.
- Browser setup and teardown completed.
- Search functionality implemented.
- Brand filter functionality implemented.
- Sorting functionality implemented.
- Price extraction implemented.
- Ascending order validation implemented.
- Product page validation implemented.

The remaining work involves refining locators to accommodate Amazon's frequently changing HTML structure and ensuring stable execution across different browser sessions.

Conclusion

Successfully developed a Selenium UI automation framework using Java, JUnit 5, and the Page Object Model (POM). Implemented automation for searching a product, applying filters and sorting, capturing prices, and validating product details. Worked on resolving dynamic locator and synchronization issues to improve the stability and reliability of the automation script